



# White Box Testing

Tim Dosen UKPL



# Teknik Pendekatan Testing yang umum dilakukan

1. Unit testing
2. Integration testing
3. Reliability testing
4. Acceptance testing
5. Certification testing
6. Regression testing



# Apa yang dites pada Unit Testing

1. Fungsi (Black Box)
2. Kode (White Box)
3. Kondisi Ekstrem dengan batasan-batasan tertentu



# Praktik System Testing Secara Umum

- Tujuan: Merakit modul menjadi suatu sistem yang bekerja dan menentukan kesiapan untuk melakukan *Acceptance Test*.
- Pelaku: Pemimpin tim atau grup tes.
- Apa yang dites:
  - Kebutuhan dan fungsi sistem.
  - Antarmuka sistem.
- Kapan selesai: Biasanya bila mayoritas kebutuhan telah sesuai dan tidak ada kesalahan mayor yang ditemukan.
- Alat bantu: Sistem pustaka dan pustaka *test case*; Generator, komparator dan simulator data testing.
- Data: Data kesalahan yang ditemukan; Test case.



## Praktik Acceptance Testing Secara Umum

- Tujuan: Mengevaluasi kesiapan untuk digunakan.
- Pelaku: Pengguna akhir atau agen.
- Apa yang dites:
  - Fungsi mayor.
  - Dokumentasi.
  - Prosedur.
- Kapan selesai: Biasanya bila pengguna telah merasa puas atau tes berjalan dengan lancar / sukses.
- Alat bantu: Komparator.
- Data: Formalitas dokumen.



# Definisi Test Case

*Test case* merupakan suatu tes yang dilakukan berdasarkan pada suatu inisialisasi, masukan, kondisi ataupun hasil yang telah ditentukan sebelumnya.



## Guna Test Case

- Untuk melakukan testing kesesuaian suatu komponen terhadap spesifikasi: *Black Box Testing*.
- Untuk melakukan testing kesesuaian suatu komponen terhadap desain - *White Box Testing*.



## White Box Testing

- Kadang disebut juga *glass box testing* atau *clear box testing*, adalah suatu metode desain *test case* yang menggunakan struktur kendali dari desain prosedural.
- Pendekatan secara internal yaitu memeriksa spesifikasi atau komponen produk.





# Kenapa Menggunakan White Box Testing?

- Banyak error dalam code yang jarang dieksekusi
- Control flow tidak dapat diprediksi secara akurat dalam black-box testing.
- Type error dapat terjadi dimana saja !



## White Box Testing

- Structural Testing or Logic-driven Testing or Glass Box Testing
- Yang dibutuhkan > **Source code**
- Menguji lebih “dekat” tentang detail prosedur perangkat lunak.
- Yang diselidiki: *logical path* (jalur logika) perangkat lunak



## Mengapa “Source code”?

- Dengan source code, dapat dilakukan pengujian tentang:
  - Structural Testing process
  - Program Logic-driven Testing
  - Design-based Testing
  - Examines the internal structure of program



## Jalur Logika

- Conditions
  - If .. Then ..
  - If .. Then .. Else ..
  - If .. Then .. Else if .. Then ..
  - Case .. Of ..
- Loop
  - While .. Do ..
  - Repeat .. Until ..
  - For .. To .. Do ..





## Contoh Test Case

**x: 1..1000;**

**Generate test cases  
to cover each statement**

```
1  INPUT-FROM-USER(x);  
   If (x <= 300) {  
2      INPUT-FROM-FILE(BALANCE);  
      If (x <= BALANCE)  
3          GiveMoney x;  
4          else Print "You don't have $x in your account!!"  
   else  
5       Print "You cannot withdraw more than $300";  
6  Eject Card;
```



# Keuntungan White Box

Menghasilkan program yang benar dan sempurna 100%, karena:

- Mengerjakan seluruh keputusan logika
- Mengerjakan seluruh loop (sesuai batas)
- Menjamin seluruh jalur independen dalam modul dikerjakan minimal 1x
- Mengerjakan seluruh data internal yang menjamin validitas

### Syarat:

- Mendefinisikan semua logical path
- Membangun kasus untuk pengujian
- Mengevaluasi hasilnya
- Menguji secara **menyeluruh**



## Kekurangan

- Pengujian secara menyeluruh justru menimbulkan masalah sumber daya
- Program yang kecil bisa menghasilkan banyak sekali jalur logika



### Kelebihan White Box Testing

- Semua jalur (*path*) yang independen/terpisah dapat dites setidaknya sekali tes.
- Semua logika keputusan dapat dites dengan jalur yang salah dan atau jalur yang benar.
- Semua *loop* dapat dites terhadap batasannya dan ikatan operasionalnya.
- Semua struktur internal data dapat dites untuk memastikan validitasnya.



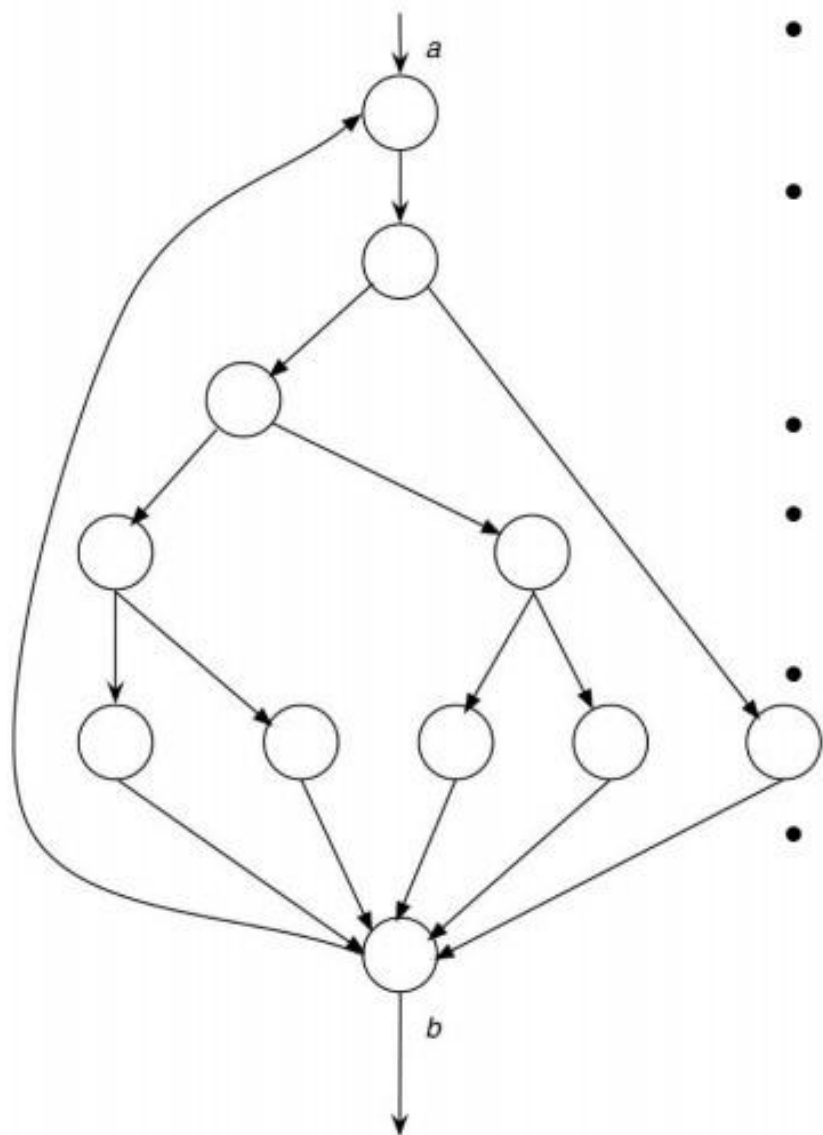


## Flow Graph (Alur Logika)

- **Nodes** (titik), mewakili pernyataan (atau sub program) yang akan ditinjau saat eksekusi program.
- **Edges** (anak panah), mewakili jalur alur logika program untuk menghubungkan satu pernyataan (atau sub program) dengan yang lainnya.
- **Branch nodes** (titik cabang), titik-titik yang mempunyai lebih dari satu anak panah keluaran.
- **Branch edges** (anak panah cabang), anak panah yang keluar dari suatu cabang
- **Paths** (jalur), jalur yang mungkin untuk bergerak dari satu titik ke lainnya sejalan dengan keberadaan arah anak panah.



## UJI KUALITAS PERANGKAT LUNAK



- Terdapat 1 (satu) buah loop yang berulang sebanyak 20 kali
- Di dalamnya terdapat nested if yang terdiri dari 4 (empat) pernyataan if..then..else
- Terdapat 5 jalur
- Asumsi keputusan saling independen
- Jumlah logic path =  $5^{20} + 5^{19} + 5^{18} + \dots + 5^0$
- Setara  $10^{14}$  jalur  
 $10^{14} = 100.000.000.000$  (seratus triliun) logic paths



# Masih “Kekurangan”

- Setiap jalur harus diuji secara “manual” untuk dibuktikan kebenarannya.
- Manual:
  - Menulis
  - Mengeksekusi
  - Memverifikasi hasil
- 1 jalur logika = 1 kegiatan manual
- Jika 1 manual dilakukan selama 5 menit, Berapa lama waktu yang diperlukan untuk pengujian?



“Bukankah *black box testing* jauh lebih cepat dan lebih mungkin dilakukan?”

- Jawaban terletak pada ketidak- sempurnaan perangkat lunak:
  - **Adanya** kesalahan logika (logic errors)  
 $65 < x \leq 75 >$  if  $(x < 65)$  AND  $(x \leq 75)$  then...
  - **Adanya** ketidaksesuaian asumsi (incorrect assumptions)  
1 bulan = 30 hari > bulan Februari? Juli? dst...
  - **Adanya** kesalahan menulis kode (typographical errors)  
 $\text{volume} := P * L * T; > \text{polume} := P * L * T; ..$



# Mungkinkah dilakukan pengujian White Box?

- Ya!
- Tidak dilakukan secara menyeluruh.
- Cukup dilakukan pada jalur logika yang penting.
- Kombinasikan dengan *black box testing*.



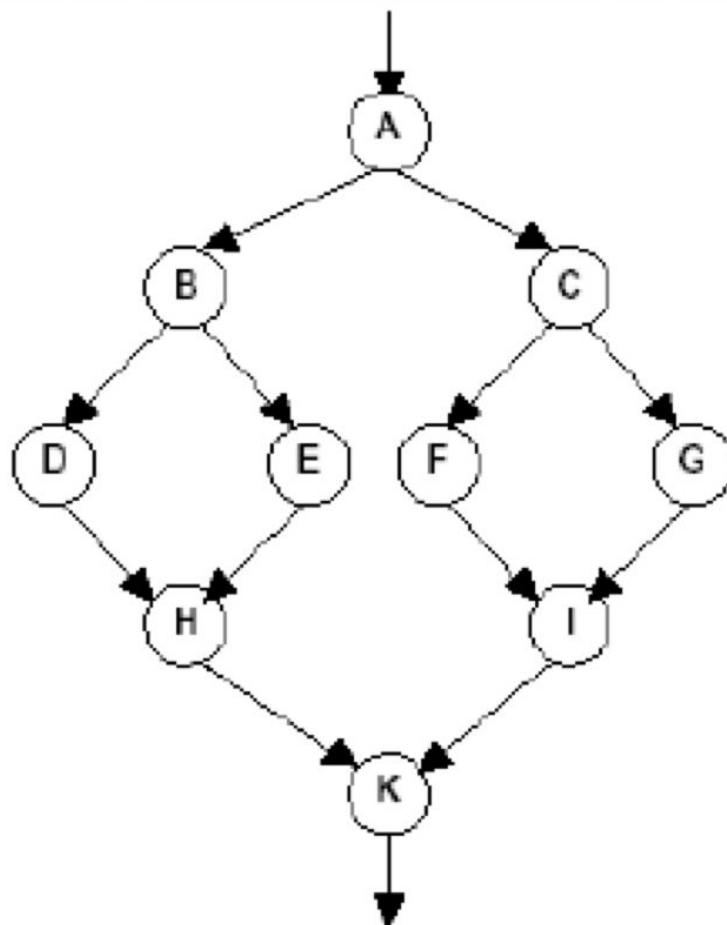
# Metode White Box | Basis path testing

- Diusulkan oleh Tom McCabe pada tahun 1976
- Digunakan untuk mendapatkan ukuran kompleksitas logika
- Ukuran ini dijadikan sebagai panduan untuk menentukan jalur-jalur utama untuk dieksekusi.



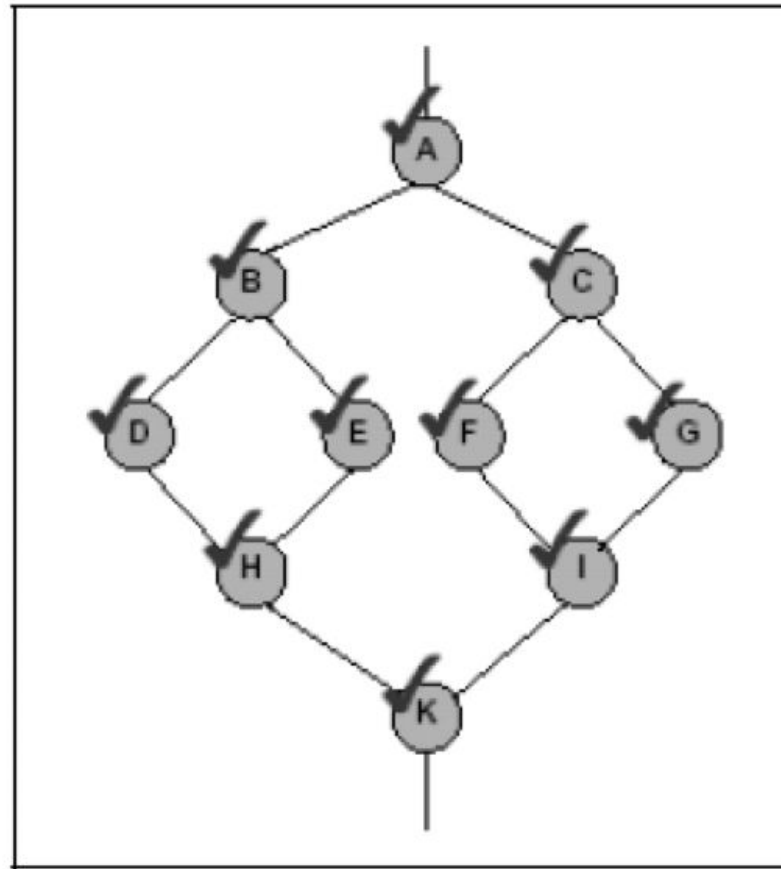
# UJI KUALITAS PERANGKAT LUNAK

```
if A then
  if B then
    D
  else
    E
  end if;
  H
else
  if C then
    F
  else
    G
  end if;
  I
end if
```





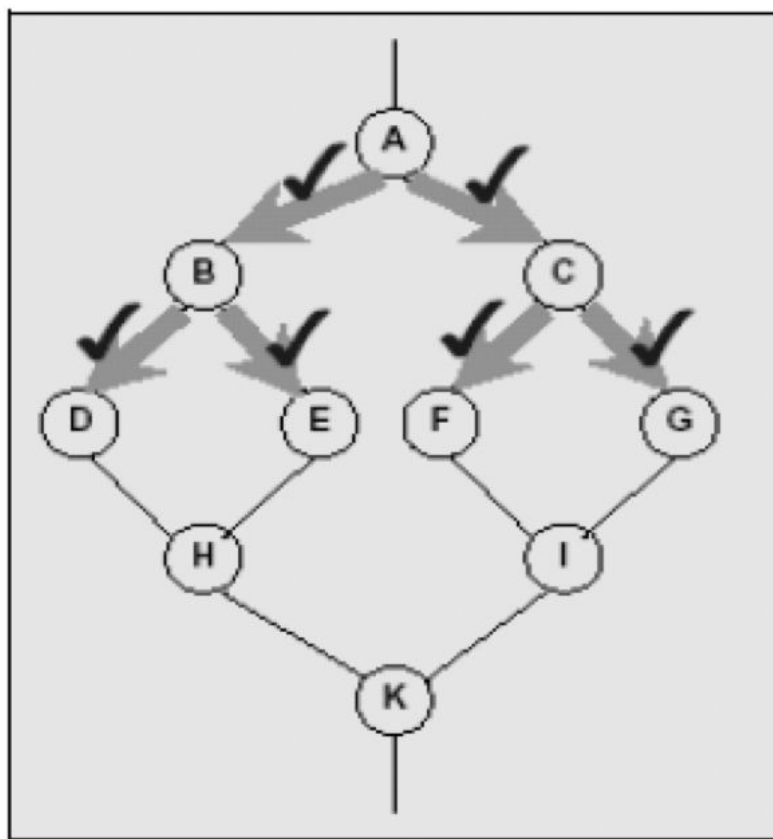
## Cakupan Pernyataan





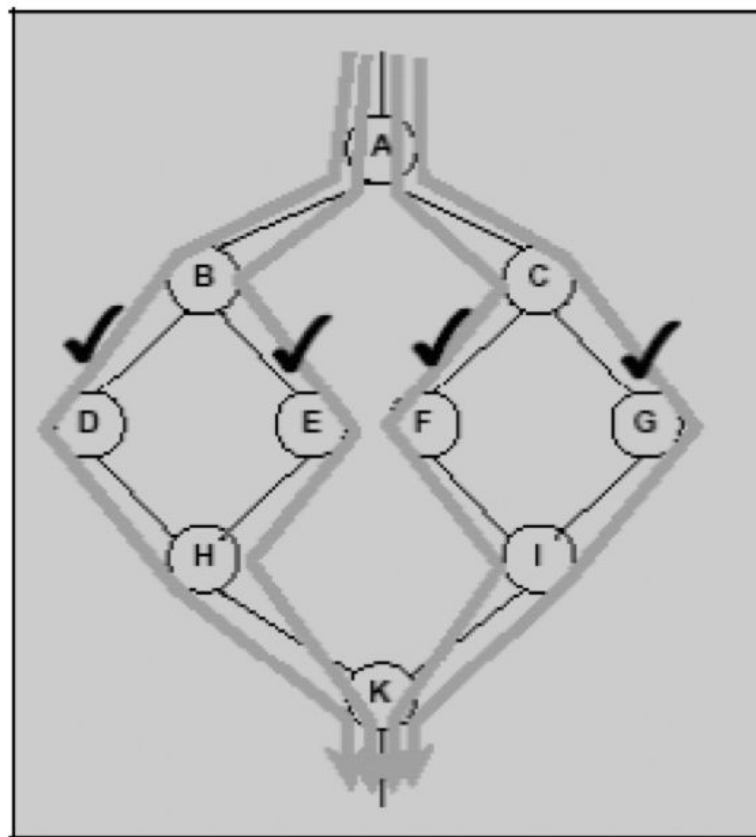


## Cakupan Cabang





## Cakupan Jalur





# Basic Path Testing (Tom McCabe)

Branch testing atau control-flow testing

Hadir dalam 2 pola:

1. **Zero Path:** Jalur penghubung yang tidak penting atau jalur pintas yang ada pada suatu sistem
2. **One Path:** Jalur penghubung yang penting atau berupa proses pada suatu sistem



## Basic Path Testing (Tujuan)

- Melakukan pengukuran kompleksitas logika dari rancangan prosedur dan menggunakannya sebagai panduan menentukan path yang dieksekusi
- Basic set akan dieksekusi oleh setiap statement paling tidak sekali
- Isu
  - Flow Graph Notation
  - Cyclomatic Complexity
- Deriving Test Cases
- Graph Matrices



## Notasi Flow Graph

Pada flow graph:

- Panah disebut *edges* menggambarkan flow of control
- Lingkaran disebut *nodes*, menggambarkan satu atau lebih aksi
- Area yang dibatasi oleh *edges* dan *nodes* disebut *regions*
- *Predicate Nodes* adalah *nodes* yang mengandung kondisi

Setiap procedural design dapat ditranslasikan ke flow graph notation



## Notasi



Lingkaran (flow graph node)  
Menyatakan satu atau beberapa statement prosedural

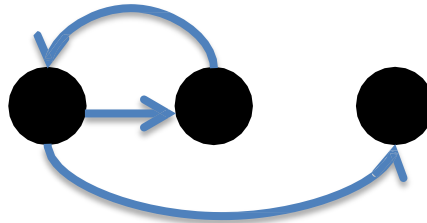


Panah (edges / links)  
Menyatakan aliran kendali (sama dengan panah flowchart)

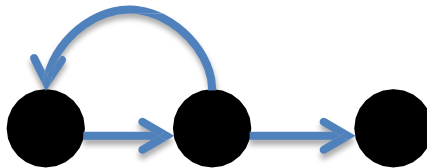
**Sequence**



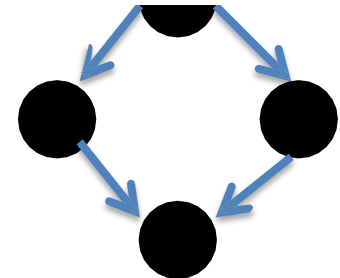
**While**



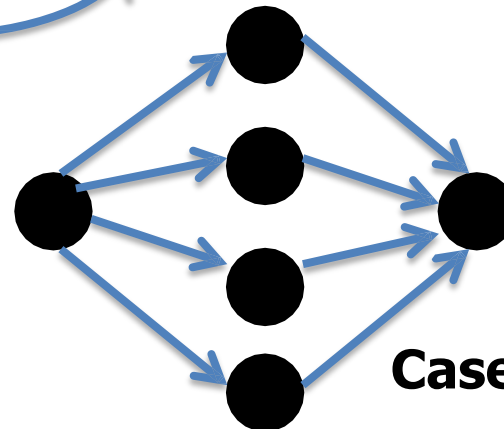
**Until**



**IF**

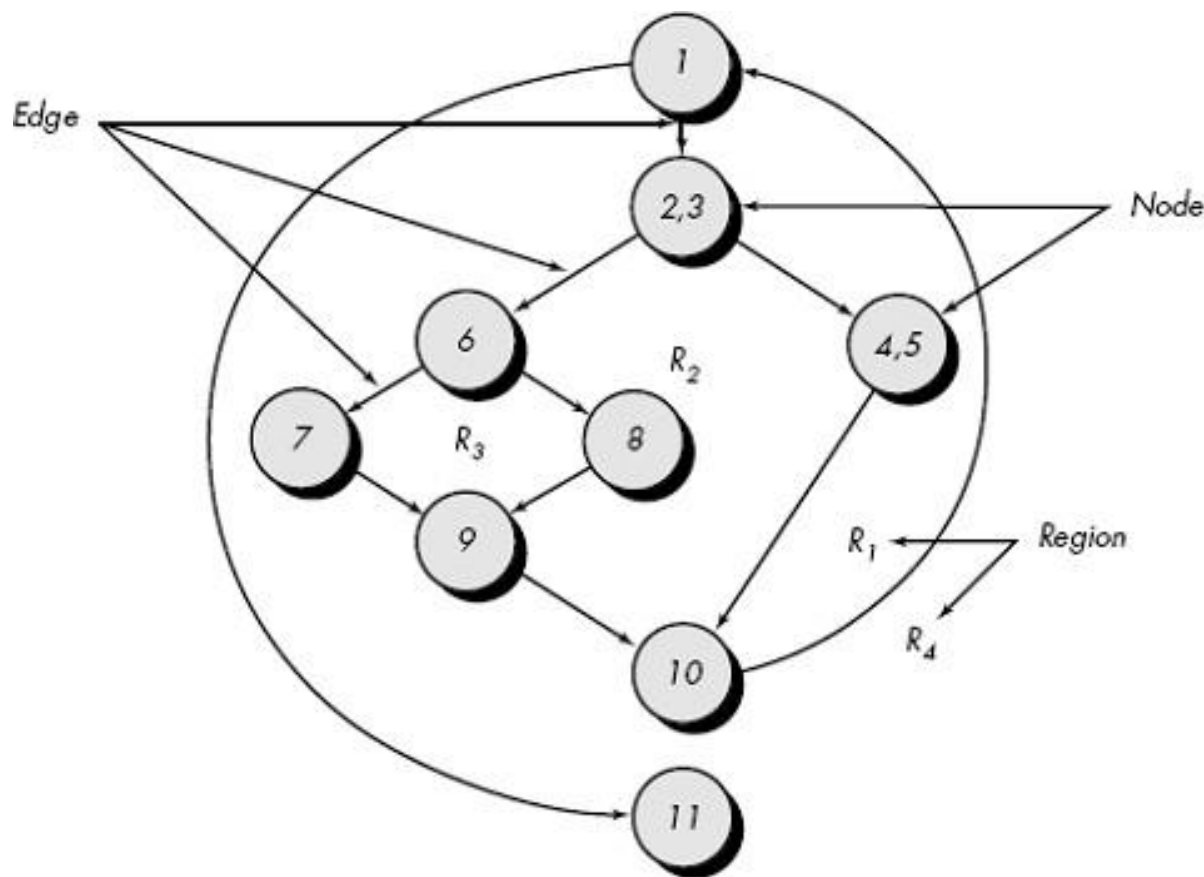


**Case**





## Regions

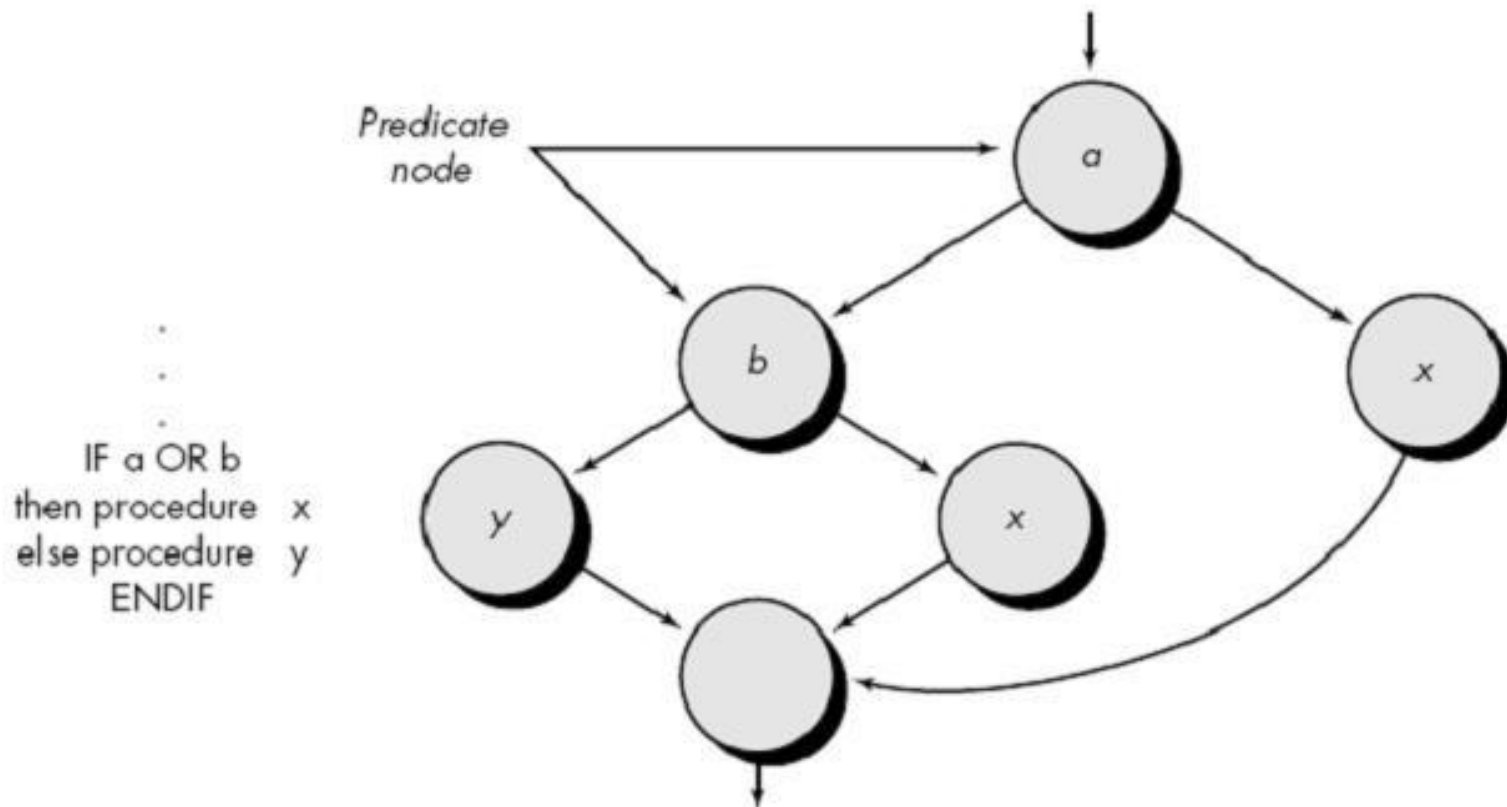


- Adalah suatu area yang dibatasi oleh edges dan nodes
- Pada saat menghitung regions, area di luar graph ikut ditambahkan



## Compound Conditions

- Kondisi majemuk terjadi apabila satu atau lebih dari operator Boolean (AND, OR, NOR, NAND) muncul pada pernyataan kondisi.
- Setiap *node* yang terdapat kondisi disebut ***predicate node***

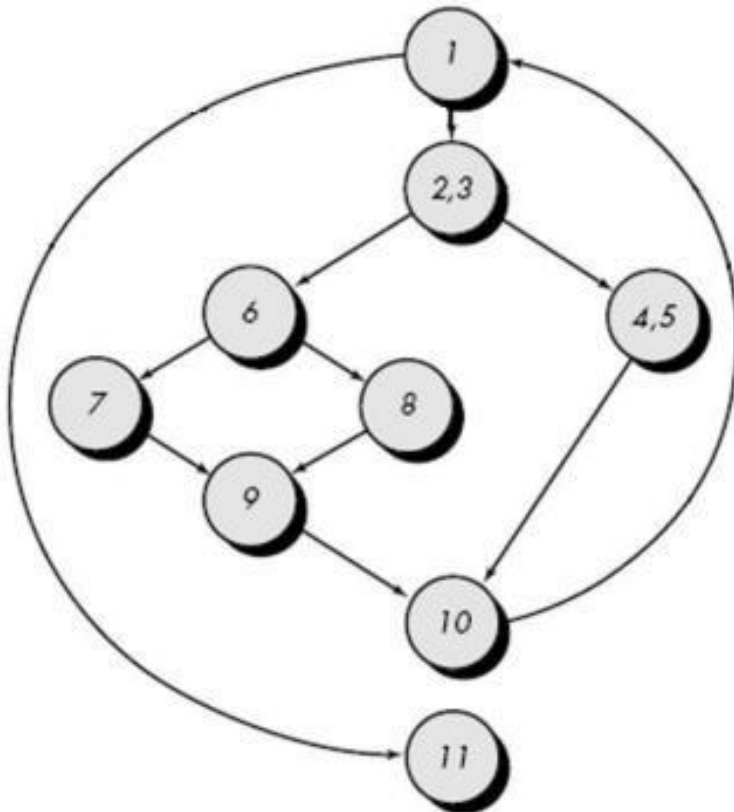






## Independent Path

- Adalah jalur yang melintasi minimal satu kumpulan pernyataan baru atau sebuah kondisi baru pada program.



Path 1 : 1 – 11

Path 2 : 1 – 2 – 3 – 4 – 5 – 10 – 1 – 11

Path 3 : 1 – 2 – 3 – 6 – 8 – 9 – 10 – 1 – 11

Path 4 : 1 – 2 – 3 – 6 – 7 – 9 – 10 – 1 – 11

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 **bukan**  
Independent path, karena merupakan  
kombinasi dari *paths* yang sudah  
didefinisikan.

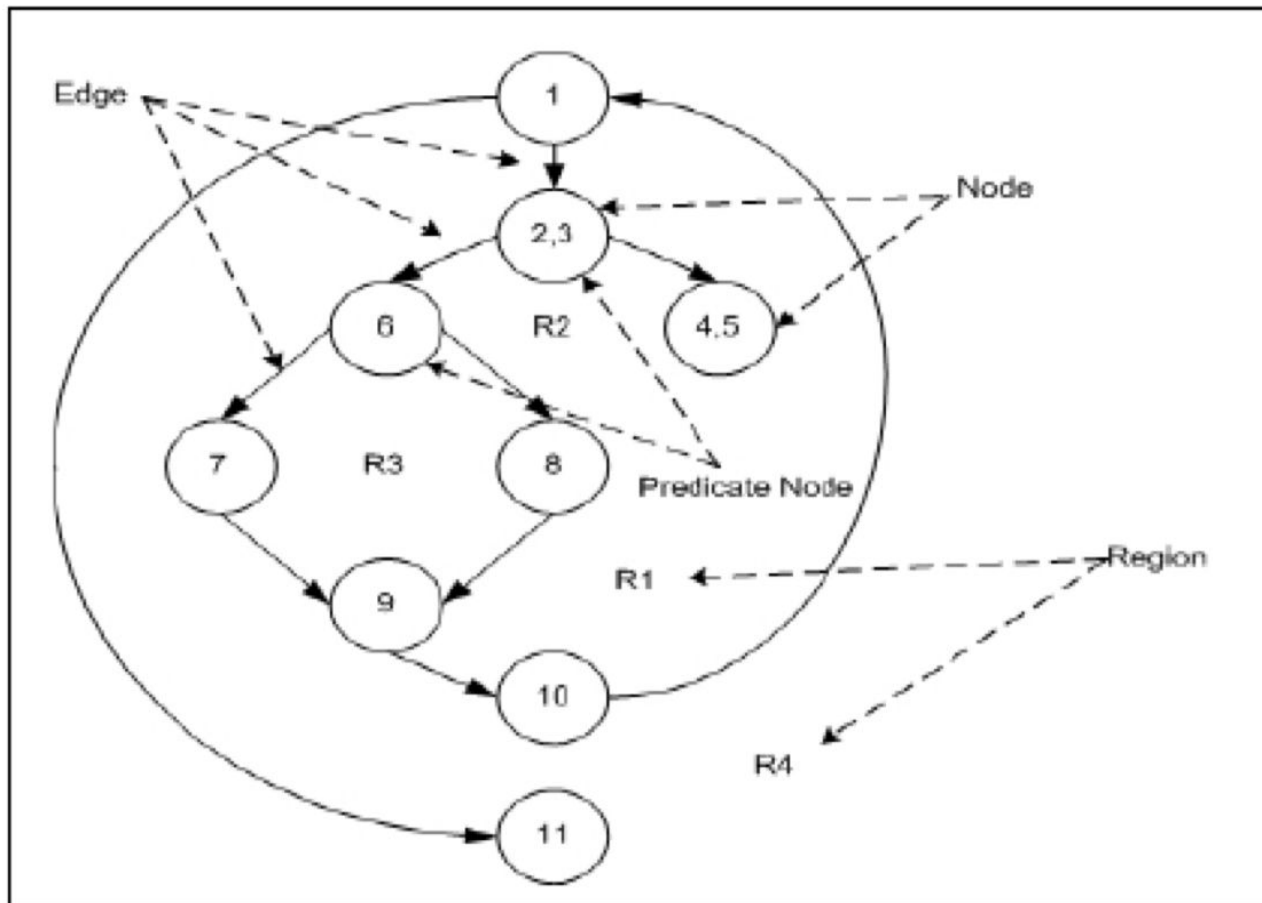


# Cyclomatic Complexity

- Memberikan ukuran kuantitatif dari kompleksitas logika.
- Nilainya memberikan jumlah dari independent path dalam basis set dan upper bound dari jumlah test untuk memastikan bahwa setiap statement dieksekusi minimal satu kali.
- Independent path adalah setiap path pada program yang mengenalkan paling tidak satu set baru statement yang sedang diproses atau kondisi baru (contoh: *edge* baru)



## Flow graph untuk Cyclomatic Complexity





# Cyclomatic Complexity

- Untuk menentukan banyaknya *independent path* yang merupakan *basis path*

- Rumus:

$$V(G) = E - N + 2$$

$$V(G) = P + 1$$

dimana:

E : Edge

N : Node

P : Predicate node



## UJI KUALITAS PERANGKAT LUNAK

- Jalur independen adalah tiap jalur pada program yang memperlihatkan 1 kelompok baru dari pernyataan proses atau kondisi baru.

$$[\text{Region / Complexity}] V(G) = E (\text{edges}) - N (\text{nodes}) + 2$$

$$V(G) = 11 - 9 + 2 = 4$$

$$V(G) = P (\text{predicate node}) + 1$$

$$V(G) = 3 + 1 = 4$$

Berdasarkan urutan alurnya, didapatkan suatu kelompok basis *flow graph* :

- Jalur 1 : 1-11
- Jalur 2 : 1-2-3-4-5-10-1-11
- Jalur 3 : 1-2-3-6-7-9-10-1-11
- Jalur 4 : 1-2-3-6-8-9-10-1-11



# Tahapan Test dengan Cyclomatic Complexity

- Gunakan desain atau kode sebagai dasar, gambarlah *flow graph*
- Berdasarkan *flow graph*, tentukan *cyclomatic complexity*
- Tentukan kelompok basis dari jalur independen secara linier
- Siapkan *test cases* yang akan melakukan eksekusi dari tiap jalur dalam kelompok basis

Catatan: Beberapa path mungkin hanya dapat dieksekusi sebagai bagian dari test yang lain.



# Contoh pengujian White BoX

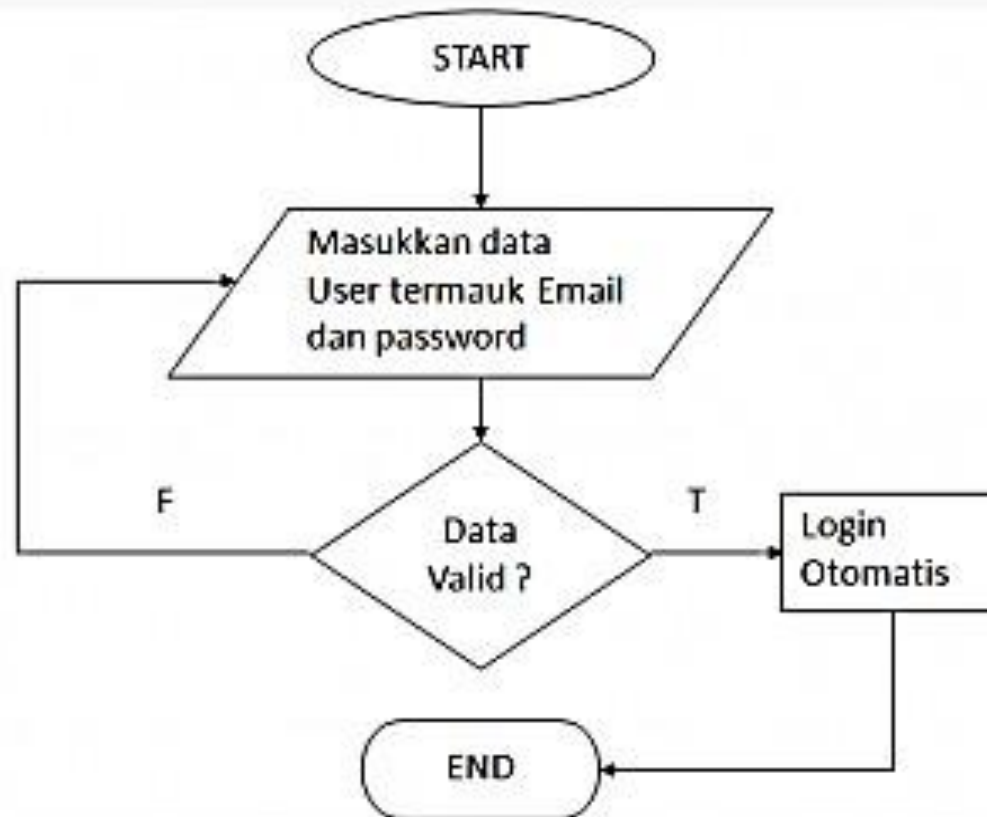
Fastncheap merupakan sebuah toko yang khusus menjual Khusus barang elektronik, mulai komputer, LCD, GPS, Laptop, dan asesoris lainnya. Fastncheap juga menyediakan toko atau website secara online. Di dalam website ini terdapat beberapa kategori dimana di setiap topik diberikan penjelasan secara rinci baik spesifikasi, kelebihan dan kelemahan terhadap produk yang dikehendaki sehingga menjadi bahan pertimbangan sebelum membeli produk tersebut.

Toko Fast & Cheap telah menggunakan Sistem Web Aplikasi Penjualan Online dimana produk disana sudah bisa dibeli secara online. Sistem disini harus bisa menampilkan Informasi Penjualan Komputer pada Toko Toko Fast & Cheap (FNC) secara online yang mempunyai beberapa cabang di Indonesia



## Contoh pengujian White Box

Seperti yang diatas, pertama-tama kami menguji form registrasi anggota

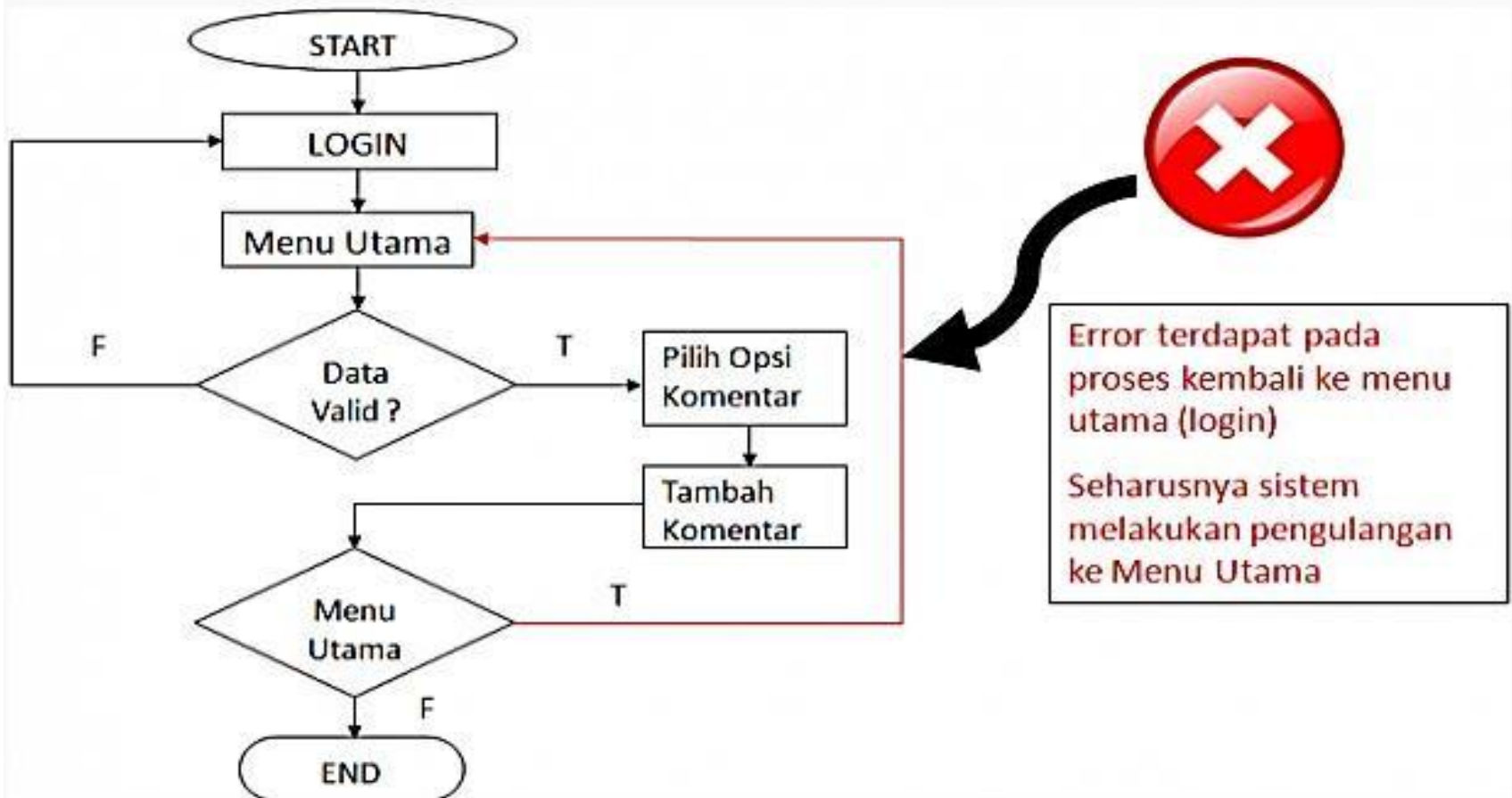






## Contoh pengujian White Box

Yang kedua kami menguji form tambah komentar





## Contoh pengujian White Box

screenshot:

Terima kasih atas komentar yang telah anda berikan pada kami.

[Close Window](#)

Fungsi tombol "Close Window" setelah memberikan komentar tidak berjalan sesuai fungsinya.

Diperkirakan link yang terdapat pada Tulisan "Close Window" sama dengan link pada posting komentar sehingga halaman tidak berpindah



## Contoh pengujian White Box

Yang ketiga kami menguji form shopping cart





## Contoh pengujian White Box

screenshot :

| Shopping cart                                     |             |     |                 |        |
|---------------------------------------------------|-------------|-----|-----------------|--------|
| Item Name                                         | Price       | Qty | Total           | Delete |
| ASYNAP VIDEO CAPTURE BOX W/AUDIO USB (AS-EZ-CAP1) | Rp. 299.000 | 900 | Rp. 269.100.000 |        |
|                                                   |             |     | +               |        |
| Sub Total                                         |             |     | Rp. 269.100.000 |        |
| <a href="#">[ Hitung Ulang ]</a>                  |             |     |                 |        |

Tidak terdapat validasi jumlah stok produk terhadap database sehingga bisa memasukkan angka jumlah produk yang sangat besar



## Contoh pengujian White Box

### Maintenance yang bisa dilakukan

- Pembersihan, pengawasan, dan pengecekan komentar yang masuk selama proses update produk
- Pengecekan kebenaran dan kecocokan database sistem terhadap produk yang ditampilkan
- Perbaikan tampilan sistem web agar lebih mudah diakses
- Pengecekan terhadap domain yang digunakan
- Pengecekan keamanan program (pembelian secara curang atau order dari Negara tertentu yang ilegal) yang digunakan sistem seiring perkembangan jaman
- Pengecekan terhadap pihak penerima pembayaran online
- Peningkatan kualitas layanan searching (pencarian)

Terlalu sering maintenance juga tidak baik karena berdampak buruk terhadap kenyamanan pelanggan





## Contoh pengujian White Box

### Fitur-fitur yang bisa ditambahkan

- Forum per topik, sehingga terdapat pertimbangan-pertimbangan dalam hal menentukan pilihan dalam membeli produk yang sedang di bahas dalam forum tersebut
- Terdapat fasilitas diskon untuk member dengan ketentuan sedemikian rupa sehingga bisa memperbanyak pelanggan.
- Fitur pengganti bahasa. Sehingga pelanggan dari dalam negeri dan luar negeri bisa juga menggunakan web tersebut.
- Fitur Cetak dokumen bagi pelanggan yang ingin mencetak informasi tentang produk ataupun di forum (jika sudah ada)
- Sistem agar bisa diakses lewat mobile
- Penambahan fitur facebook, twitter, blog dsb. Untuk dapat lebih memudahkan dalam hal promosi.



## Tugas kelompok (2-3 orang)!

1. Membuat/mencari program sederhana **contoh:**  
Program untuk menentukan bilangan real negatif atau positif, bilangan prima, bilangan ganjil genap (**Source code**) bisa menggunakan program yang lain.
2. Gambarkan flow graphnya
3. Tuliskan path independennya (jalur)
4. Hitung Cyclomatic Complexity nya
5. Tugas diupload di Spada dalam bentuk \*.pdf secara mandiri dan dipresensasikan minggu depan